NavAI

Hands-Free Voice Travel Companion

Product & Technical Design Document · v1.0 · June 2026

---

1. Overview

NavAI is a native mobile voice assistant for hands-free driving. You say a wake word, ask a natural question, and hear a spoken answer — no screen interaction required. It understands Indian languages, follows up on previous questions, and gets smarter about your regular routes over time.

Target user: Anyone commuting in Indian cities — Bangalore specifically — who wants route, traffic, weather, and place information while keeping their eyes on the road.

Platform: Native iOS and Android via React Native + Expo. Not a web app — background mic access and native GPS are required for the core experience.

> *Personal project scope. This is not a production SaaS product. API usage stays within free tiers. The goal is a working, usable app and a strong portfolio piece demonstrating voice pipeline + agentic tool calling on a real problem.*

2. Features

Phase 1 — Core (build first)

2.1 Wake word activation

Say the wake word ("Hey Nav" or "Navigate") and the app activates from any state — screen off, music playing, app in background. A Siri-style animated overlay appears and the mic opens. No button tap needed.

- Porcupine (Picovoice) runs as a background service, on-device, no network call
- Custom wake word trained in Picovoice Console — free for personal/non-commercial use
- ~50ms response time from word completion to overlay appearing
- Overlay dismissed automatically after response, or by saying "cancel"

2.2 ETA and traffic queries

Ask about travel time to any destination, with live traffic awareness.

- "How long to Koramangala in this traffic?"
- "Is the highway or the service road faster right now?"
- "How long will it take if I leave in 20 minutes instead?"

Responses are concise and spoken: "About 35 minutes via Hosur Road — Silk Board is heavy, the service road saves you 8 minutes."

2.3 Nearby place search

Find places near your current location by type. Uses real-time GPS, not a fixed point.

- "Is there a petrol bunk near me?"
- "Find a pharmacy on my route"
- "Any open restaurants nearby?"

Response includes name, distance, direction, and open/closed status. Follow-up "How do I get there?" triggers directions automatically from context.

2.4 Weather on route

Check weather at your current location or destination, with commute-relevant framing.

- "Will it rain before I reach home?"
- "What's the weather at the airport?"
- "Should I carry an umbrella?"

2.5 Multi-turn context

The agent remembers what was said earlier in the same drive session. Pronouns and references resolve correctly across turns.

- "What about avoiding the highway?" — knows the destination from the previous turn
- "How far is it from there?" — resolves "there" from context
- "Is it open right now?" — refers to the last place mentioned

Session context resets when you end the drive or the app goes to background for more than 30 minutes.

2.6 Multilingual support

Hindi, Tamil, Telugu, Kannada, and English work naturally. Hinglish (code-switching) is handled correctly.

- "Yaar, Silk Board pe kitna traffic hai?"
- "Koramangala yaav route better?" (Kannada-English)

Sarvam saaras:v3 with translate mode handles transcription and converts regional languages to English before the LLM. TTS responds in the same language the question was asked in.

Phase 2 — Enhancements

2.7 Saved places

Set home, office, and custom locations by voice. "Save this as my office." Subsequent queries resolve "home" and "office" without coordinates.

2.8 Proactive traffic alerts

When a destination is active, the agent monitors the route in the background and speaks up unprompted: "Traffic building on your route — alternate via Hosur Road saves 12 minutes." Configurable sensitivity threshold.

2.9 Smart on-route detours

"Add a coffee stop on my way to the airport." Finds places that are genuinely on the route with minimal detour, not just nearby. Compares added time: "Starbucks on Outer Ring Road adds 4 minutes vs 14 minutes for the one in Koramangala."

Out of scope

- Turn-by-turn navigation (Google Maps / Apple Maps handles this — NavAI is the voice layer on top, not a navigation replacement)
- Full duplex continuous conversation — push-to-talk is the model for driving safety
- Calendar, music control, or calling integrations — keep the app focused on driving
- Multi-user or cloud account system — all data local to the device

3. Architecture

3.1 System overview

NavAI has two main parts: the mobile app (React Native) and a backend server (FastAPI + Python). They communicate over WebSocket for audio streaming and HTTP for TTS response audio. The backend holds all AI logic; the app is a thin voice I/O shell.

> *Architectural principle (inherited from MASAI / MIS Orchestrator): all AI service calls happen in the backend. The mobile app never calls Sarvam, Google Maps, or any AI API directly. It sends audio/transcript to the backend and receives text or audio back.*

Layer	What it does	Key technology
Mobile app	Wake word, mic capture, GPS, UI overlay, audio playback	React Native + Expo

Layer	What it does	Key technology
Transport	Real-time audio streaming between app and backend	LiveKit WebRTC
Audio processing	Noise suppression, VAD, resampling	WebRTC built-in + Silero VAD
STT	Speech to text — Indian languages + English	Sarvam saaras:v3
Agent brain	Intent parsing, tool routing, response generation	LangGraph + Gemini 2.5 Flash
Tools	Maps, places, weather, session state	Google Maps API, OpenWeatherMap
TTS	Text to speech — Indian languages + English	Sarvam TTS

3.2 Voice pipeline — step by step

1. User says wake word ("Hey Nav") — Porcupine detects on-device, fires event to React Native layer
2. App activates overlay UI, opens mic, begins streaming audio to backend via LiveKit room
3. Backend receives audio frames — WebRTC built-in noise suppression already applied by LiveKit
4. Silero VAD runs on incoming frames server-side, detects end-of-speech, triggers STT
5. Audio chunk sent to Sarvam saaras:v3 WebSocket STT with translate mode — returns English transcript
6. Transcript + current GPS coordinates + session context passed to LangGraph StateGraph
7. Gemini 2.5 Flash reads query, decides which tools to call, executes them via ReAct loop
8. Tool results (route data, place info, weather) returned to Gemini, which generates spoken response text
9. Response text sent to Sarvam TTS — streaming audio chunks returned to backend
10. Backend streams TTS audio back to app via LiveKit audio track
11. App plays audio through device speaker / bluetooth, with audio ducking (music lowers while agent speaks)

> *No pipeline framework (Pipecat, LiveKit Agents) is used. Push-to-talk means one discrete round trip per query — a custom asyncio WebSocket handler in FastAPI is sufficient and simpler.*

3.3 How the agent picks the right API

The agent is a LangGraph ReAct agent running Gemini 2.5 Flash. It receives the transcript, current location, and session context, and has access to five LangChain tools. Gemini reads the tool descriptions and decides which to call — there is no hardcoded if/else routing.

Tool name	API it calls	When Gemini calls it
get_directions	Google Maps Directions API	ETA, route, traffic queries, navigation
find_nearby_places	Google Places API Nearby Search	"near me" queries, place type searches
get_weather	OpenWeatherMap current + forecast	Weather queries at current or destination location
geocode_place	Google Maps Geocoding API	Converting place names to coordinates before other tool calls
get_session_context	Local session state (Redis / in-memory)	Resolving 'home', 'there', 'it', destination from prior turns

Current GPS location is not a tool. It is injected into LangGraph state automatically with every query, so all tools that need it receive it as a parameter from state without an extra LLM step.

Example: "Will it rain before I reach home?"

12. Gemini identifies this as a weather query requiring the destination "home"
13. Calls get_session_context(key="home_location") — retrieves saved home coordinates
14. Calls get_weather(location=home_coords, hours=2) — gets 2-hour forecast
15. Generates spoken response: "Light rain expected in 40 minutes at your area — you should reach before it starts."

Example: "What about avoiding the highway?" (follow-up)

16. LangGraph state has destination from previous turn — Gemini reads full conversation history
17. Calls get_session_context to confirm stored destination
18. Calls get_directions with avoid_highways=true
19. Compares and responds: "Via the service road it adds 7 minutes but avoids Silk Board entirely."

4. Full Tech Stack

4.1 Mobile app

Component	Technology	Notes
Framework	React Native + Expo	Expo managed workflow — no Xcode/Android Studio needed for dev
Wake word	Porcupine (Picovoice)	Free personal use, custom wake word, on-device, React Native SDK

Component	Technology	Notes
Audio transport	LiveKit React Native SDK	WebRTC, same LiveKit you use at MoveInSync
GPS	expo-location	Background location updates, high accuracy mode
Animations	React Native Reanimated	Siri-style waveform overlay, smooth transitions
Audio ducking	expo-av / AVAudioSession	Lowers music while agent speaks, routes through bluetooth
State management	Zustand	Lightweight, simple — no Redux overhead for a personal project

4.2 Backend

Component	Technology	Notes
API server	FastAPI + Python 3.11	Async WebSocket handler for voice pipeline
Agent framework	LangGraph	StateGraph with ReAct agent loop, same as MASAI
LLM	Gemini 2.5 Flash	Free tier (15 RPM, 1M tokens/day), fast TTFB
VAD	Silero VAD	Server-side, Python, detects end of speech in audio frames
STT	Sarvam saaras:v3	Streaming WebSocket, translate mode for Indian languages
TTS	Sarvam TTS	Streaming audio output, Hindi + English voices
Session state	Redis (or in-memory dict for v1)	Stores destination, saved places, conversation history per session
Media server	LiveKit (self-hosted Docker or LiveKit Cloud free tier)	WebRTC rooms for audio transport

4.3 External APIs

API	Used for	Free tier	Cost at personal scale
Google Maps Directions	ETA, routes, traffic	$200/month credit	Effectively free
Google Places API	Nearby search	$200/month credit	Effectively free
Google Geocoding	Place name to coordinates	$200/month credit	Effectively free
OpenWeatherMap	Weather + forecast	1,000 calls/day	Free

API	Used for	Free tier	Cost at personal scale
Sarvam AI (STT + TTS)	Voice pipeline	Free personal tier	Free
Gemini 2.5 Flash	LLM reasoning	15 RPM, 1M tok/day	Free
LiveKit Cloud	WebRTC media server	100GB bandwidth/month	Free
Porcupine	Wake word detection	1 custom wake word	Free (personal)

> *Total estimated monthly API cost: Rs 0–200. Set a Google Cloud billing alert at $10 to catch any accidental loops early.*

5. Data Flow

5.1 Audio message payload

When a voice query completes (VAD end-of-speech), the backend assembles the following state object before invoking LangGraph:

Field	Source	Description
transcript	Sarvam STT	English text of what the user said
current_location	Mobile GPS (sent with audio)	{ lat, lng } of user at time of query
session_id	App-generated UUID per drive	Ties multi-turn context together
conversation_history	LangGraph state / Redis	Prior turns in this session for context resolution
saved_places	Local device storage	Home, office, custom saved locations
language_hint	Sarvam STT metadata	Detected language of the query for TTS voice selection

5.2 LangGraph state schema

The LangGraph StateGraph carries this typed state across nodes:

```
class NavState(TypedDict):
    transcript: str          # current query
    current_location: dict      # { lat, lng }
    session_id: str          # drive session identifier
```

```
conversation_history: list   # prior turns
saved_places: dict           # home, office, custom
tool_results: list           # outputs from tool calls
response_text: str           # final response for TTS
language: str              # detected language code
```

6. UX Design

6.1 Voice overlay states

State	Visual + behaviour
Idle	App is in background or screen off. Porcupine listening silently. No UI shown.
Wake word detected	Screen lights up (if off). Semi-transparent overlay slides up from bottom. Animated waveform pulses. Haptic feedback fires.
Listening	Waveform animates to mic input amplitude in real time. "Listening..." label. User speaks.
Processing	Waveform transitions to a thinking animation (dots / spinner). "Got it..." label. VAD fired, STT and LLM running.
Responding	Waveform animates to TTS audio output amplitude. Agent speaks. Transcript of response shown as text in overlay.
Done	Overlay fades out after 1.5s. App returns to previous state.
Error	Overlay shows brief error message ("Couldn't hear clearly" / "No internet"). Fades out after 2s.

6.2 Main app screen

The app has a minimal home screen — most interaction happens through voice without touching the screen.

- Large wake word activation button as fallback (for when voice isn't practical)
- Current destination card (if a route is active) showing ETA and traffic status
- Last 5 queries shown as a scrollable history — tap any to repeat it
- Settings: wake word on/off, language preference, saved places management

6.3 Response design principles

LLM system prompt enforces these rules for all spoken responses:

- Maximum 2 sentences for simple queries, 3 for complex comparisons
- No markdown, no lists, no bullet points — everything written as natural spoken language

- Lead with the answer, then the detail: "35 minutes via Hosur Road — Silk Board is heavy right now." Not: "Due to heavy traffic at Silk Board junction, travel time is approximately 35 minutes."
- Use cardinal directions and landmarks, not street numbers: "turn left after the Nandi flyover", not "turn left at 12.97°N"
- Numbers spoken naturally: "about 35 minutes", "roughly 2 kilometres", not "34.7 minutes"

7. Build Plan

Week 1 — Backend voice pipeline

- Set up FastAPI project structure, LiveKit self-hosted (Docker)
- Build WebSocket endpoint: receive audio frames, run Silero VAD, call Sarvam STT
- Test with a simple echo server: speak, get transcript back
- Add Sarvam TTS: text in, audio stream out
- End-to-end test: speak → transcript → TTS echo response

Week 2 — LangGraph agent + tools

- Set up LangGraph StateGraph with NavState schema
- Implement get_directions tool (Google Maps Directions API)
- Implement find_nearby_places tool (Google Places API)
- Implement get_weather tool (OpenWeatherMap)
- Implement geocode_place tool (Google Geocoding API)
- Implement get_session_context tool (in-memory session dict)
- Wire ReAct agent: transcript in → tool calls → response text out
- Test: type queries directly to backend, verify correct tools fire

Week 3 — React Native app shell

- Set up Expo project, install LiveKit React Native SDK
- Build basic voice overlay UI with React Native Reanimated waveform
- Wire LiveKit client: connect to room, stream mic audio, receive audio track
- Add expo-location GPS: send coordinates with each query
- Add audio ducking (expo-av)
- Test end-to-end: speak on phone → backend → agent → TTS → hear response

Week 4 — Wake word + polish

- Integrate Porcupine React Native SDK
- Train custom wake word in Picovoice Console

- Wire wake word event to overlay activation
- Add session context and multi-turn memory
- Multilingual TTS: detect language from Sarvam metadata, switch voice
- Test in a real car environment — tune VAD silence threshold for road noise
- Performance pass: measure voice-to-voice latency, optimise if > 1.5s

Phase 2 additions (after core is solid)

- Saved places: voice-set home and office, persist to device storage
- Proactive traffic alerts: background location polling + route monitoring
- On-route detour suggestions: waypoint-aware place search

8. Latency Targets

Stage	Target	Bottleneck
Wake word to overlay appear	< 100ms	Porcupine on-device
VAD end-of-speech to STT start	< 50ms	Server-side Silero
STT final transcript	< 800ms after speech ends	Sarvam saaras:v3
LLM first token	< 400ms after transcript	Gemini Flash TTFB
Tool calls (1–2 per query)	< 400ms total	Google Maps / OWM API
TTS first audio chunk	< 300ms after response text	Sarvam TTS streaming
Total voice-to-voice (target)	< 2 seconds	Sum of above stages

> *2 seconds voice-to-voice is the target for v1. Sub-1.5s is achievable with streaming optimisations in v2 (start TTS before LLM finishes). The VAD silence threshold (default 800ms) is the single biggest tuning lever.*

9. Failure Handling

Voice pipeline failures

- No speech detected after 8s: session times out, overlay dismisses, Porcupine resumes listening
- STT error from Sarvam: backend returns error event, app shows "Couldn't hear clearly — try again"
- Network unavailable: app detects offline state before activating overlay, shows "No internet connection"
- LiveKit connection drop: automatic reconnect with exponential backoff, user notified if > 3s

Agent failures

- Google Maps API error: agent responds "Couldn't get route info right now" without crashing
- LLM timeout: 5s hard timeout on Gemini call, fallback response: "I couldn't process that — try again"
- Tool returns empty result (no places found): agent responds naturally: "No petrol bunks found within 2km"
- Ambiguous query: agent asks one clarifying question rather than guessing: "Did you mean the Koramangala 5th Block or 1st Block?"

10. Open Questions

#	Question	Options
1	Wake word: train a custom word or use a built-in Porcupine keyword?	A) Train "Hey Nav" custom B) Use built-in "Hey Siri" style — Porcupine has "Jarvis", "Porcupine" as built-ins
2	Session state storage: in-memory dict (v1) or Redis?	A) In-memory — simpler, resets on server restart B) Redis — survives restarts, overkill for personal project
3	LiveKit: self-hosted Docker or LiveKit Cloud free tier?	A) Docker locally for dev, Cloud for production B) Cloud only — less ops overhead
4	VAD silence threshold: 800ms or tune for car noise?	Test in a real car first — road noise may require 1000–1200ms to avoid false end-of-speech
5	Response language: always English, or match query language?	A) Always English — simpler B) Match query language via Sarvam TTS voice selection — better UX